Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №6

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

на тему: «Унарные и бинарные операции над графами»

Выполнили ст.гр. 25ВВВ3:
Волков И.Р.
Итянина В.А.
Приняли:
к.т.н., доцент Юрова О.В.
к.т.н, ст. преподаватель, Деев М.В.

Пенза 2026

Цель работы: изучить унарные и бинарные операции над графами, научится реализовывать операции отождествления вершин, стягивания ребра, расщепления вершины, объединения, пересечения, кольцевой суммы и декартова произведения графов.

Ход работы

Унарные и бинарные операции над графами – это действия, с помощью которых можно изменять граф или получать новый граф из одного или нескольких исходных графов.

- Унарная операция – выполняется над одним графом

- Удаление вершины – из графа убирают одну вершину и все ребра, связанные с ней.
- Удаление ребра – из графа убирают одно или несколько рёбер.
- Добавление вершины или ребра – в граф вносят новые элементы.

- Бинарная операция – выполняется над двумя графами

- Объединение графов (объединяем вершины и рёбра обоих графов, если вершина или ребро встречается в обоих графах, оно учитывается один раз)
- Пересечение графов (оставляем только те вершины и рёбра, которые есть в обоих графах)
- Разность графов (оставляем элементы первого графа, которых нет во втором)
- Концевая сумма графов (оставляем рёбра, которые присутствуют только в одном из двух графов, но не в обоих одновременно)

Отличия:

Признак	**Унарная операция**	**Бинарная операция**
Сколько исходных графов?	Один	Два
Пример	Удаление вершины	Объединение графов
Что происходит?	Изменяем один граф	Сопоставляем или объединяем два графа

Задание 1: сгенерируйте (используя генератор случайных чисел) две матрицы M_1 и M_2 смежности неориентированных помеченных графов G_1, G_2. Выведите сгенерированные матрицы на экран. *Для указанных графов преобразуйте представление матриц смежности в списки смежности. Выведите полученные списки на экран.

Код:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

#define MAXV 100


void print_matrix(int n, int mat[MAXV][MAXV]) {
    printf("Матрица смежности (%d вершин):\n", n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }
}


void print_adj_list(int n, int mat[MAXV][MAXV]) {
    printf("Списки смежности:\n");
    for (int i = 0; i < n; i++) {
        printf("v%d: ", i + 1);
        int first = 1;
        for (int j = 0; j < n; j++) {
            if (mat[i][j]) {
                if (!first) printf(", ");
                printf("v%d", j + 1);
                first = 0;
            }
        }
        printf("\n");
    }
}


void generate_graph(int n, int mat[MAXV][MAXV], double p) {

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            mat[i][j] = 0;


    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if ((double)rand() / RAND_MAX < p) {
                mat[i][j] = 1;
                mat[j][i] = 1;
            }
        }
    }
}
```

```
int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));

    int n;
    printf("Введите число вершин для G1 и G2: ");
    scanf_s("%d", &n);

    if (n <= 0 || n > MAXV) {
        printf("Недопустимое число вершин. Допустимо от 1 до
%d.\n", MAXV);
        return 1;
    }

    int G1[MAXV][MAXV], G2[MAXV][MAXV];

    generate_graph(n, G1, 0.3);
    generate_graph(n, G2, 0.3);

    printf("\n Граф G1 \n");
    print_matrix(n, G1);
    printf("\n");
    print_adj_list(n, G1);

    printf("\n Граф G2 \n");
    print_matrix(n, G2);
    printf("\n");
    print_adj_list(n, G2);

    return 0;
}
```

Результат работы программы представлен на рисунке 1.

```
Введите число вершин для G1 и G2: 12

 Граф G1
Матрица смежности (12 вершин):
0 0 0 1 0 0 0 1 0 0 1 1
0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 1 0 1
1 0 0 0 0 0 1 0 1 0 0 1
0 0 0 0 0 1 1 1 0 0 1 0
0 0 0 0 1 0 0 0 0 1 1 0
0 0 0 1 1 0 0 0 0 0 0 1
1 0 0 0 1 0 0 0 1 1 0 0
0 1 0 1 0 0 0 1 0 0 1 0
0 0 1 0 0 1 0 1 0 0 0 0
1 0 0 0 1 1 0 0 1 0 0 1
1 0 1 1 0 0 1 0 0 0 1 0

Списки смежности:
v1: v4, v8, v11, v12
v2: v9
v3: v10, v12
v4: v1, v7, v9, v12
v5: v6, v7, v8, v11
v6: v5, v10, v11
v7: v4, v5, v12
v8: v1, v5, v9, v10
v9: v2, v4, v8, v11
v10: v3, v6, v8
v11: v1, v5, v6, v9, v12
v12: v1, v3, v4, v7, v11

 Граф G2
Матрица смежности (12 вершин):
0 0 1 0 0 1 0 0 0 0 1 1
0 0 1 0 0 0 1 1 0 0 0 0
1 1 0 1 0 0 0 1 0 0 0 1
0 0 1 0 1 0 0 0 0 0 0 0
0 0 0 1 0 1 1 0 0 0 1 0
1 0 0 0 1 0 0 0 0 1 1 0
0 1 0 0 1 0 0 1 1 0 0 0
0 1 1 0 0 0 1 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 1 0 0 0 0 1 0
1 0 0 0 1 1 0 0 0 1 0 1
1 0 1 0 0 0 0 0 0 0 1 0

Списки смежности:
v1: v3, v6, v11, v12
v2: v3, v7, v8
v3: v1, v2, v4, v8, v12
v4: v3, v5
v5: v4, v6, v7, v11
v6: v1, v5, v10, v11
v7: v2, v5, v8, v9
v8: v2, v3, v7
v9: v7
v10: v6, v11
v11: v1, v5, v6, v10, v12
v12: v1, v3, v11
```

Рис.1 – результат работы программы к заданию 1.

Задание 2: для матричной формы представления графов выполните операцию: 1. отождествление вершин; 2. стягивание ребра; 3. расщепления вершины. Номера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнение операции выведите на экран. *Для преставления графов в виде списков смежности выполните операцию: 1. отождествления вершин; 2. стягивания ребра; 3. расщепления вершины. Номера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнения выведите на экран.

Код:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#include <windows.h>

#define MAXV 100

void print_matrix(int n, int mat[MAXV][MAXV]) {
    printf("Матрица смежности (%d вершин):\n", n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) printf("%d ", mat[i][j]);
        printf("\n");
    }
}

void print_adj_list(int n, int mat[MAXV][MAXV]) {
    printf("Списки смежности:\n");
    for (int i = 0; i < n; i++) {
        printf("v%d: ", i + 1);
        for (int j = 0; j < n; j++)
            if (mat[i][j]) printf("v%d ", j + 1);
        printf("\n");
    }
}

void generate_graph(int n, int mat[MAXV][MAXV], double p) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) mat[i][j] = 0;
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if ((double)rand() / RAND_MAX < p)
                mat[i][j] = mat[j][i] = 1;
}

//отождествление вершин u и v
void identify(int n, int mat[MAXV][MAXV], int u, int v, int* nn,
int res[MAXV][MAXV]) {
```

```
    if (u == v) { printf("Вершины должны быть различны!\n"); *nn
= -1; return; }
    int map[MAXV], idx = 0;
    for (int i = 0; i < n; i++) map[i] = (i == u || i == v) ? -1
: idx++;
    int z = idx, total = idx + 1;

    for (int i = 0; i < total; i++)
        for (int j = 0; j < total; j++) res[i][j] = 0;

    for (int i = 0; i < n; i++) {
        if (map[i] == -1) continue;
        for (int j = 0; j < n; j++) {
            if (map[j] == -1) continue;
            if (mat[i][j]) res[map[i]][map[j]] = 1;
        }
    }
    for (int w = 0; w < n; w++) {
        if (map[w] == -1) continue;
        if (mat[w][u] || mat[w][v]) {
            res[map[w]][z] = 1;
            res[z][map[w]] = 1;
        }
    }
    *nn = total;
}

// Стягивание ребра
void contract(int n, int mat[MAXV][MAXV], int u, int v, int* nn,
int res[MAXV][MAXV]) {
    if (u == v || mat[u][v] == 0) {
        printf("Ребра между вершинами нет!\n");
        *nn = -1; return;
    }
    identify(n, mat, u, v, nn, res);
}

// Расщепление вершины v
void split(int n, int mat[MAXV][MAXV], int v, int* nn, int
res[MAXV][MAXV]) {
    int total = n + 1, w = n;
    for (int i = 0; i < total; i++)
        for (int j = 0; j < total; j++) res[i][j] = 0;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            if (i != v && j != v) res[i][j] = mat[i][j];

    for (int i = 0; i < n; i++)
        if (i != v && mat[v][i]) {
            res[w][i] = 1;
            res[i][w] = 1;
        }
```

```
    res[v][w] = res[w][v] = 1;
    *nn = total;
}

int main() {
    setlocale(LC_ALL, "Russian");
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    srand(time(NULL));

    int n;
    printf("Введите число вершин: ");
    scanf_s("%d", &n);
    if (n <= 0 || n > MAXV) { printf("Неверное число!\n");
return 1; }

    int G1[MAXV][MAXV], G2[MAXV][MAXV];
    generate_graph(n, G1, 0.3);
    generate_graph(n, G2, 0.3);

    printf("\n G1 \n");
    print_matrix(n, G1); print_adj_list(n, G1);
    printf("\n G2 \n");
    print_matrix(n, G2); print_adj_list(n, G2);

    int g, op, u, v, nn;
    int res[MAXV][MAXV];

    while (1) {
        printf("\nГраф (1-G1, 2-G2, 0-выход): ");
        scanf_s("%d", &g);
        if (g == 0) break;
        if (g != 1 && g != 2) { printf("Неверно!\n"); continue;
}

        int (*G)[MAXV] = (g == 1) ? G1 : G2;

        printf("Операция (1-отожд., 2-стягивание, 3-
расщепление): ");
        scanf_s("%d", &op);

        if (op == 1) {
            printf("Вершины u v: ");
            scanf_s("%d %d", &u, &v);
            identify(n, G, u - 1, v - 1, &nn, res);
        }
        else if (op == 2) {
            printf("Вершины u v: ");
            scanf_s("%d %d", &u, &v);
            contract(n, G, u - 1, v - 1, &nn, res);
        }
        else if (op == 3) {
            printf("Вершина v: ");
```

```
            scanf_s("%d", &v);
            split(n, G, v - 1, &nn, res);
        }
        else {
            printf("Неверная операция!\n");
            continue;
        }

        if (nn > 0) {
            printf("\nРезультат:\n");
            print_matrix(nn, res);
            print_adj_list(nn, res);
        }
    }
    return 0;
}
```

Результат работы программы представлен на рисунке 2.

```
Введите число вершин: 3

 G1
Матрица смежности (3 вершин):
0 1 1
1 0 0
1 0 0
Списки смежности:
v1: v2 v3
v2: v1
v3: v1

 G2
Матрица смежности (3 вершин):
0 1 0
1 0 1
0 1 0
Списки смежности:
v1: v2
v2: v1 v3
v3: v2

Граф (1-G1, 2-G2, 0-выход): 2
Операция (1-отожд., 2-стягивание, 3-расщепление): 3
Вершина v: 2

Результат:
Матрица смежности (4 вершин):
0 0 0 1
0 0 0 1
0 0 0 1
1 1 1 0
Списки смежности:
v1: v4
v2: v4
v3: v4
v4: v1 v2 v3

Граф (1-G1, 2-G2, 0-выход):
```

Рис.2 – результат работы программы к заданию 2.

Задание 3: для матричной формы представления графов выполните операцию: 1. объединение $G = G_1 G_2$; 2. пересечения $G = G_1 G_2$; 3. концевой суммы $G = G_1 + G_2$. Результат выполнения операции выведите на экран.

Код:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));
    int n, G1[50][50], G2[50][50], R[50][50];

    printf("Введите n: ");
    scanf_s("%d", &n);

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            G1[i][j] = G2[i][j] = 0;

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++) {
            if (rand() % 2) G1[i][j] = G1[j][i] = 1;
            if (rand() % 2) G2[i][j] = G2[j][i] = 1;
        }

    printf("\nG1:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) printf("%d ", G1[i][j]);
        printf("\n");
    }
    printf("G2:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) printf("%d ", G2[i][j]);
        printf("\n");
    }

    printf("\nОбъединение:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            R[i][j] = G1[i][j] | G2[i][j];
            printf("%d ", R[i][j]);
        }
        printf("\n");
    }

    printf("\nПересечение:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
```

```
            R[i][j] = G1[i][j] & G2[i][j];
            printf("%d ", R[i][j]);
        }
        printf("\n");
    }

    printf("\nКольцевая сумма:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            R[i][j] = G1[i][j] ^ G2[i][j];
            printf("%d ", R[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Результат работы программы представлен на рисунке 3.

```
Введите n: 8

G1:
0 1 1 0 1 1 1 0
1 0 1 0 0 0 0 1
1 1 0 0 0 0 0 1
0 0 0 0 1 1 0 0
1 0 0 1 0 0 1 0
1 0 0 1 0 0 0 1
1 0 0 0 1 0 0 1
0 1 1 0 0 1 1 0
G2:
0 0 0 1 0 1 0 0
0 0 0 0 0 1 0 1
0 0 0 1 1 1 1 1
1 0 1 0 1 1 1 0
0 0 1 1 0 0 0 0
1 1 1 1 0 0 0 0
0 0 1 1 0 0 0 1
0 1 1 0 0 0 1 0

Объединение:
0 1 1 1 1 1 1 0
1 0 1 0 0 1 0 1
1 1 0 1 1 1 1 1
1 0 1 0 1 1 1 0
1 0 1 1 0 0 1 0
1 1 1 1 0 0 0 1
1 0 1 1 1 0 0 1
0 1 1 0 0 1 1 0

Пересечение:
0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 1
0 0 0 0 0 0 0 1
0 0 0 0 1 1 0 0
0 0 0 1 0 0 0 0
1 0 0 1 0 0 0 0
0 0 0 0 0 0 0 1
0 1 1 0 0 0 1 0

Кольцевая сумма:
0 1 1 1 1 0 1 0
1 0 1 0 0 1 0 0
1 1 0 1 1 1 1 0
1 0 1 0 0 0 1 0
1 0 1 0 0 0 1 0
0 1 1 0 0 0 0 1
1 0 1 1 1 0 0 0
0 0 0 0 0 1 0 0
```

Рис.3– результат работы программы к заданию 3.

Задание 4: для матричной формы преставления графов выполните операцию декартов произведения графов $G = G_1 \times G_2$. Результат выполнения операции выведите на экран.

Код:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

#define MAX 20
#define MAXP 400

void generate(int n, int m[MAX][MAX]) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            m[i][j] = 0;

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (rand() % 2)
                m[i][j] = m[j][i] = 1;
}

void print(int n, int m[MAX][MAX]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%d ", m[i][j]);
        printf("\n");
    }
}

int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));

    int n1, n2;
    printf("Введите размеры G1 и G2 (n1 n2): ");
    scanf("%d %d", &n1, &n2);

    if (n1 <= 0 || n2 <= 0 || n1 > MAX || n2 > MAX) {
        printf("Недопустимые размеры!\n");
        return 1;
    }

    int G1[MAX][MAX], G2[MAX][MAX];
    generate(n1, G1);
    generate(n2, G2);
```

```
    printf("\nG1:\n");
    print(n1, G1);

    printf("\nG2:\n");
    print(n2, G2);

    int N = n1 * n2;
    int P[MAXP][MAXP] = { 0 };

    //декартово произведение
    for (int i = 0; i < n1; i++) {
        for (int k = 0; k < n2; k++) {
            int a = i * n2 + k;
            for (int j = 0; j < n1; j++) {
                for (int l = 0; l < n2; l++) {
                    int b = j * n2 + l;
                    if (a == b) continue;

                    //условие смежности
                    if ((i == j && G2[k][l]) || (k == l &&
G1[i][j]))
                        P[a][b] = 1;
                }
            }
        }
    }

    printf("\nG1 x G2 (%d вершин):\n", N);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            printf("%d ", P[i][j]);
        printf("\n");
    }

    return 0;
}
```

Результат работы программы представлен на рисунке 4.

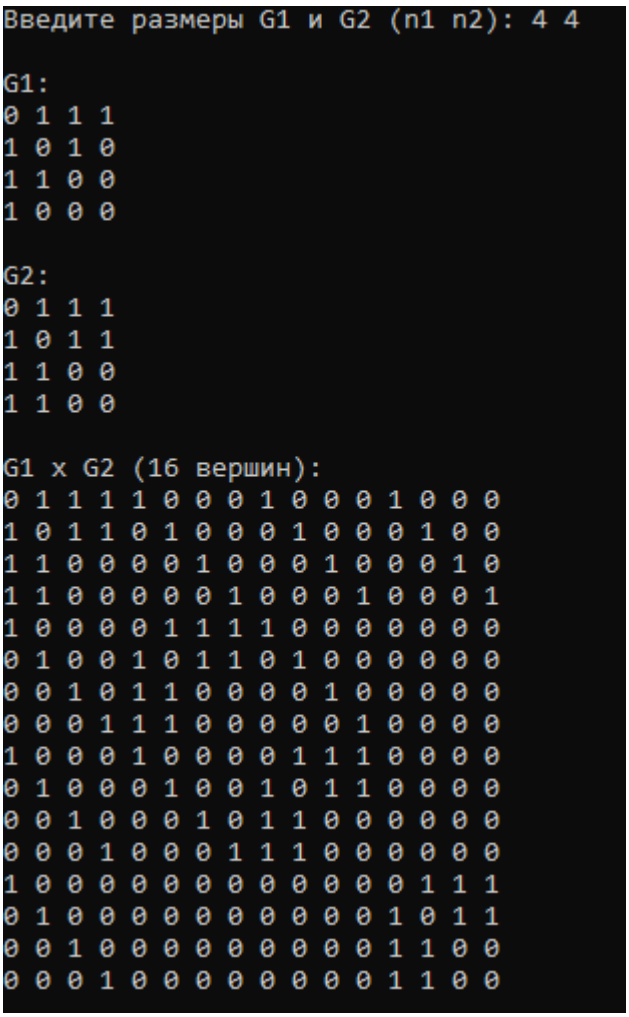


Рис.4 – результат работы программы к заданию 4.

Вывод: в ходе лабораторной работы были сгенерированы два графа (G_1 и G_2), построены их матрицы смежности и списки смежности. Для графов выполнены унарные операции: отождествление вершин, стягивание ребра и расщепление вершины, а также бинарные операции: объединение, пересечение, кольцевая сумма и декартово произведение.